

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 2
Проектирование и технический дизайн API

Выполнила:

Персук Виктория

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

1. Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
2. Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
3. Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

Приложение для бронирования столиков в ресторанах. Формат реализации API – REST API.

Функциональные требования:

1. Управление пользователями
 - a. Регистрация пользователя по email и паролю
 - b. Аутентификация пользователя (логин)
 - c. Получение информации о текущем пользователе
 - d. Редактирование профиля пользователя
 - e. Хранение роли пользователя (admin / customer)
2. Управление ресторанами
 - a. Создание ресторана авторизованным пользователем
 - b. Автоматическое назначение роли owner пользователю, создавшему ресторан
 - c. Редактирование информации о ресторане (только owner/admin)
 - d. Удаление ресторана (admin)
 - e. Просмотр списка ресторанов
3. Модерация ресторанов
 - a. Присвоение ресторану статуса
 - i. Pending – при создании
 - ii. Verified – после проверки (admin)
 - iii. Rejected – при отклонении
 - b. Отображение в поиске только ресторанов со статусом verified

- с. Возможность администратору изменять статус ресторана
- 4. Поиск и фильтрация ресторанов
 - а. Система должна обеспечивать поиск ресторанов с возможностью фильтрации по:
 - i. Городу
 - ii. Типу кухни
 - iii. Ценовой категории
- 5. Управление персоналом ресторана
 - а. Назначение менеджеров ресторана владельцем
 - б. Наличие нескольких менеджеров у одного ресторана
 - с. Разграничение прав между owner и manager
- 6. Меню ресторана
 - а. Просмотр меню ресторана
 - б. Добавление блюд (owner)
 - с. Редактирование блюд (owner)
 - д. Удаление блюд (owner)
- 7. Бронирование столиков
 - а. Создание бронирования пользователем
 - б. Создание бронирования менеджером или владельцем ресторана
 - с. Отмена бронирования
 - д. Изменение бронирования
 - е. Нельзя создать бронирование, если столик уже занят на выбранное время

Ограничения:

- Пользователь может изменять или отменять бронирование:
 - только своё
 - не позднее чем за 3 часа до времени бронирования
- Владелец и менеджер могут:
 - управлять любыми бронированиями своего ресторана
 - без ограничения по времени

Нельзя создать бронирование, если:

- столик уже занят на выбранное время
- гостей больше, чем может вместить столик

8. История бронирований
 - a. Получение списка бронирований пользователя
 - b. Получение списка бронирований ресторана (для owner/manager)
9. Отзывы
 - a. Добавление отзыва пользователем
 - b. Просмотр отзывов ресторана
 - c. Редактирование отзыва автором
 - d. Удаление отзыва автором или администратором

Нефункциональные требования:

1. Производительность
 - a. Время ответа API не должно превышать 300 мс (P95)
 - b. Система должна поддерживать одновременную работу множества пользователей
 - c. Должны использоваться индексы для ускорения запросов
2. Масштабируемость
 - a. Backend должен быть stateless
 - b. Система должна поддерживать горизонтальное масштабирование
 - c. Возможность разбиения на микросервисы
3. Надёжность
 - a. Система должна обеспечивать корректную обработку ошибок
 - b. Должна быть обеспечена целостность данных
 - c. Операции бронирования должны выполняться в транзакциях
4. Безопасность
 - a. Пароли должны храниться в хешированном виде (bcrypt)
 - b. Использование JWT для аутентификации
 - c. Проверка прав доступа (RBAC)
5. Консистентность данных
 - a. Исключение двойного бронирования одного столика на одно время
 - b. Валидация входных данных
 - c. Ограничения на уровне БД (CHECK, FK, UNIQUE)
6. Удобство сопровождения

- a. Код должен быть модульным и расширяемым
 - b. Чёткое разделение слоёв (controller / service / repository)
 - c. Возможность добавления новых ролей и функционала
7. Логирование и мониторинг
- a. Логирование запросов и ошибок
 - b. Возможность отслеживания проблем в системе

Routes для API

1. Авторизация
 - a. POST /auth/register – Регистрация нового пользователя
 - b. POST /auth/login – Вход, получение токенов
 - c. POST /auth/refresh – Обновление access-токена
2. Пользователи
 - a. GET /users/me – Получить профиль текущего пользователя
 - b. PATCH /users/me – Обновить профиль текущего пользователя
 - c. GET /users/me/reservations – История бронирований текущего пользователя
3. Кухни
 - a. GET /cuisines – Список всех кухонь
4. Рестораны
 - a. GET /restaurants – Список верифицированных ресторанов (фильтры, сортировка, пагинация)
 - b. POST /restaurants – Создать ресторан
 - c. GET /restaurants/{id} – Получить ресторан по ID
 - d. PATCH /restaurants/{id} – Обновить ресторан (owner / admin)
 - e. DELETE /restaurants/{id} – Удалить ресторан (admin)
 - f. PATCH /restaurants/{id}/verify – Подтвердить ресторан (admin)
 - g. PATCH /restaurants/{id}/reject – Отклонить ресторан (admin)
5. Персонал
 - a. GET /restaurants/{id}/staff – Список сотрудников ресторана
 - b. POST /restaurants/{id}/staff – Назначить менеджера
 - c. DELETE /restaurants/{id}/staff/{user_id} – Удалить сотрудника
6. Меню
 - a. GET /restaurants/{id}/menus – Список всех разделов меню ресторана
 - b. POST /restaurants/{id}/menus – Создать раздел меню

- c. GET /restaurants/{id}/menu – Полное меню ресторана со всеми блюдами
- d. POST /menu-items – Добавить блюдо в меню
- e. PATCH /menu-items/{id} – Обновить блюдо
- f. DELETE /menu-items/{id} – Удалить блюдо

7. Столики

- a. GET /restaurants/{id}/tables – Список столиков ресторана
- b. POST /tables – Создать столик
- c. PATCH /tables/{id} – Обновить столик
- d. DELETE /tables/{id} – Удалить столик

8. Бронирования

- a. POST /reservations – Создать бронирование
- b. GET /reservations/{id} – Получить бронирование по ID
- c. PATCH /reservations/{id} – Изменить бронирование
- d. DELETE /reservations/{id} – Отменить бронирование
- e. GET /restaurants/{id}/reservations – Список бронирований ресторана (owner / manager)

9. Отзывы

- a. GET /restaurants/{id}/reviews – Список отзывов ресторана
- b. POST /restaurants/{id}/reviews – Добавить отзыв
- c. PATCH /reviews/{id} – Обновить отзыв (автор)
- d. DELETE /reviews/{id} – Удалить отзыв (автор / admin)

10. Фотографии

- a. GET /restaurants/{id}/photos – Список фото ресторана
- b. POST /restaurants/{id}/photos – Добавить фото ресторана
- c. DELETE /photos/{id} – Удалить фото

Приложение использует RBAC, поэтому надо расписать у кого на что есть доступ

Матрица доступа

Endpoint	guest	user	manager	owner	admin
Авторизация					
POST /auth/register	+	+	+	+	+
POST /auth/login	+	+	+	+	+
POST /auth/refresh	+	+	+	+	+
Пользователь					
GET /users/me		+	+	+	+
PATCH /users/me		+	+	+	+
GET /users/me/reservations		+	+	+	+
Рестораны					
GET /restaurants	+	+	+	+	+
GET /restaurants/{id}	+	+	+	+	+
POST /restaurants		+	+	+	+
PATCH /restaurants/{id}				+	+
DELETE /restaurants/{id}					+
PATCH /restaurants/{id}/verify					+
PATCH /restaurants/{id}/reject					+
Персонал					
POST /restaurants/{id}/staff				+	+
GET /restaurants/{id}/staff			+	+	+
DELETE /restaurants/{id}/staff/{user_id}				+	+
Кухня					
GET /cuisines	+	+	+	+	+
Меню					
GET /restaurants/{id}/menus	+	+	+	+	+
POST /restaurants/{id}/menus				+	+
GET /restaurants/{id}/menu	+	+	+	+	+
POST /menu-items				+	+
PATCH /menu-items/{id}				+	+
DELETE /menu-items/{id}				+	+
Столики					
GET /restaurants/{id}/tables	+	+	+	+	+
POST /tables			+	+	+
PATCH /tables/{id}			+	+	+
DELETE /tables/{id}			+	+	+

Endpoint	guest	user	manager	owner	admin
Бронирования					
POST /reservations		+	+	+	+
GET /reservations/me		+	+	+	+
GET /restaurants/{id}/reservations			+	+	+
PATCH /reservations/{id}		+	+	+	+
DELETE /reservations/{id}		+	+	+	+
Отзывы					
POST /reviews		+	+	+	+
GET /restaurants/{id}/reviews	+	+	+	+	+
PATCH /reviews/{id}		+			+
DELETE /reviews/{id}		+			+
Фото ресторана					
GET /restaurants/{id}/photos	+	+	+	+	+
POST /restaurants/{id}/photos				+	+
DELETE /photos/{id}				+	+

* только минимально за 3 часа до начала брони

** только свой отзыв

Вывод

В ходе работы над домашним заданием, были написаны функциональные и нефункциональные требования, написаны руты для API, сделана матрица доступа для разных ролей. Была также написана документация к API на YAML в формате OpenAPI (файл на гитхабе).